

1. Garantir que comece no canto inferior esquerdo

9500

1.1 Tomar medidas máximas em passos:

=> Usou-se: `const float VELOCIDADE_X = 180.0; const float VELOCIDADE_Y = 180.0;`

- Passos em x para painel todo: 9150 passos -> com margem de segurança: 9300 (aproximadamente 1,6% a mais)

- Passos em y para painel todo: 9400 passos -> com margem de segurança: 9500 (aproximadamente 1,0% a mais)

=> Programa:

```
C/C++
#include <Arduino.h>
#include <AccelStepper.h>

#define FULLSTEP 4

// Motor Y: D12, D11, D10, D9
AccelStepper motorY(FULLSTEP, D12, D10, D11, D9);

// Motor X: D8, D7, D6, D3
AccelStepper motorX(FULLSTEP, D8, D6, D7, D3);

// =====
// INVERSÃO LÓGICA DOS EIXOS
// =====

// + x para a direita e + y para cima.
const bool INVERTER_X = false;
const bool INVERTER_Y = true;

long fisicoX(long x) { return INVERTER_X ? -x : x; }

long fisicoY(long y) { return INVERTER_Y ? -y : y; }

long logX() { return INVERTER_X ? -motorX.currentPosition() :
motorX.currentPosition(); }

long logY() { return INVERTER_Y ? -motorY.currentPosition() :
motorY.currentPosition(); }

// =====
// CONFIGURAÇÃO DE VELOCIDADE
// =====

const float VELOCIDADE_X = 180.0;
const float VELOCIDADE_Y = 180.0;
```

```

// =====
// ESTADO DO CONTROLE MANUAL
// =====

enum MovimentoManual {
    PARADO,
    MOVENDO_X_POS,
    MOVENDO_X_NEG,
    MOVENDO_Y_POS,
    MOVENDO_Y_NEG
};

MovimentoManual movimentoAtual = PARADO;

unsigned long ultimoPrint = 0;
const unsigned long INTERVALO_PRINT_MS = 300;

// =====
// FUNÇÕES AUXILIARES
// =====

void imprimirAjuda() {
    Serial.println();
    Serial.println("=====");
    Serial.println(" CALIBRACAO MANUAL DE PASSOS");
    Serial.println("=====");
    Serial.println("Comandos:");
    Serial.println("  x+  -> mover X no sentido positivo");
    Serial.println("  x-  -> mover X no sentido negativo");
    Serial.println("  y+  -> mover Y no sentido positivo");
    Serial.println("  y-  -> mover Y no sentido negativo");
    Serial.println("  s    -> parar");
    Serial.println("  p    -> imprimir posicao atual");
    Serial.println("  z    -> zerar X e Y na posicao atual");
    Serial.println("  h    -> mostrar ajuda");
    Serial.println();
    Serial.println("Modo de uso:");
    Serial.println("1) Coloque manualmente no canto inferior esquerdo.");
    Serial.println("2) Digite z para zerar.");
    Serial.println("3) Digite x+ ou y+ para mover.");
    Serial.println("4) Digite s para parar.");
    Serial.println("5) Leia X e Y no Serial.");
    Serial.println("=====");
    Serial.println();
}

void imprimirPosicao() {

```

```

    Serial.print("[POSICAO] X = ");
    Serial.print(logX());
    Serial.print(" passos | Y = ");
    Serial.print(logY());
    Serial.println(" passos");
}

void pararMotores() {
    movimentoAtual = PARADO;

    motorX.setSpeed(0);
    motorY.setSpeed(0);

    Serial.println();
    Serial.println("Movimento parado.");
    imprimirPosicao();
    Serial.println();
}

void zerarPosicaoAtual() {
    motorX.setCurrentPosition(0);
    motorY.setCurrentPosition(0);

    Serial.println();
    Serial.println("Posicao atual definida como origem:");
    Serial.println("X = 0 | Y = 0");
    Serial.println();
}

void processarComando(String comando) {
    comando.trim();
    comando.toLowerCase();

    if (comando == "x+") {
        movimentoAtual = MOVENDO_X_POS;
        motorX.setSpeed(fisicoX(VELOCIDADE_X));
        motorY.setSpeed(0);

        Serial.println();
        Serial.println("Movendo X positivo...");
    }

    else if (comando == "x-") {
        movimentoAtual = MOVENDO_X_NEG;
        motorX.setSpeed(fisicoX(-VELOCIDADE_X));
        motorY.setSpeed(0);

        Serial.println();
    }
}

```

```

        Serial.println("Movendo X negativo...");
    }

    else if (comando == "y+") {
        movimentoAtual = MOVENDO_Y_POS;
        motorY.setSpeed(fisicoY(VELOCIDADE_Y));
        motorX.setSpeed(0);

        Serial.println();
        Serial.println("Movendo Y positivo...");
    }

    else if (comando == "y-") {
        movimentoAtual = MOVENDO_Y_NEG;
        motorY.setSpeed(fisicoY(-VELOCIDADE_Y));
        motorX.setSpeed(0);

        Serial.println();
        Serial.println("Movendo Y negativo...");
    }

    else if (comando == "s") {
        pararMotores();
    }

    else if (comando == "p") {
        imprimirPosicao();
    }

    else if (comando == "z") {
        zerarPosicaoAtual();
    }

    else if (comando == "h") {
        imprimirAjuda();
    }

    else {
        Serial.println();
        Serial.print("Comando desconhecido: ");
        Serial.println(comando);
        Serial.println("Digite h para ver os comandos.");
        Serial.println();
    }
}

// =====
// SETUP

```

```
// =====

void setup() {
    Serial.begin(115200);
    delay(1000);

    motorX.setMaxSpeed(300);
    motorY.setMaxSpeed(300);

    motorX.setAcceleration(200);
    motorY.setAcceleration(200);

    motorX.enableOutputs();
    motorY.enableOutputs();

    zerarPosicaoAtual();
    imprimirAjuda();
}

// =====
// LOOP
// =====

void loop() {
    // Lê comandos do Serial Monitor
    if (Serial.available() > 0) {
        String comando = Serial.readStringUntil('\n');
        processarComando(comando);
    }

    // Mantém o motor andando conforme o comando atual
    if (movimentoAtual == MOVENDO_X_POS || movimentoAtual == MOVENDO_X_NEG) {
        motorX.runSpeed();
    }

    if (movimentoAtual == MOVENDO_Y_POS || movimentoAtual == MOVENDO_Y_NEG) {
        motorY.runSpeed();
    }

    // Imprime posição periodicamente enquanto estiver se movendo
    if (movimentoAtual != PARADO) {
        unsigned long agora = millis();

        if (agora - ultimoPrint >= INTERVALO_PRINT_MS) {
            ultimoPrint = agora;
            imprimirPosicao();
        }
    }
}
```

